

Meten.AI

An onboarding briefing: eight questions, answered

This condenses the Meten.AI technical documentation (Inge Lodder and Raul Quintana Gonzalez, 26 June 2026) into the answers needed to get productive on the project. It keeps the real terminology, because you will meet it in the code and in meetings, but explains each term the first time it appears.

One thing to know before reading the source yourself: the Word version carries **tracked changes**, and the corrections are substantial. They add an entire staging environment, a referral programme, an onboarding tutorial, the two-factor login flow, and several admissions that things do not work as first described. If Word shows "Simple Markup", you are reading an outdated document. Switch to **All Markup** or **No Markup**. This briefing follows the corrected text throughout.

The one-paragraph version. Meten.AI reads a PDF floor plan, uses an external AI to measure every room, and turns those measurements into a priced quote for a client. It is a subscription web application for Dutch construction firms, built as three separate services (browser app, API, analysis worker) hosted on Microsoft Azure, with the database and files held at Supabase and payments through Mollie.

The eight questions, and where each is answered

#	Question	Short answer	Page
1	Core workflow, upload to output	Upload and crop, analysis fires automatically, results drawn over the PDF, quote emailed	2
2	Tech stack and architecture	Three services: Next.js frontend, FastAPI backend, FastAPI worker. Python 3.11 and Node 22	3
3	How the codebase is organised	One monorepo: backend/, frontend/, worker/, infra/, docs/, .github/workflows/	5
4	The Azure environment	One resource group, two App Services, one Container App, a registry and a Key Vault - times two for staging	6
5	Access you need	Seven systems. The documentation names the resources but not who grants them	7
6	Where customer data is stored	Supabase: PostgreSQL in Ireland for records, an uploads bucket for files. Separation is enforced in code only	8
7	Testing and release process	No automated testing is described anywhere. Release is a push to a branch	9
8	The harder components	The worker's threading and coordinate maths, then tenant isolation, then the deployment split	10

A short glossary closes the document, covering the terms that make the source hard to read.

QUESTION 1

The core workflow, from upload to final output

A user uploads a floor plan PDF, crops it and sets the scale. Analysis starts immediately, runs on a separate service for up to ten minutes, and returns every room as an outline with an area in square metres. Those areas are priced from the firm's own price list and become a quote PDF, downloaded or emailed to the client.

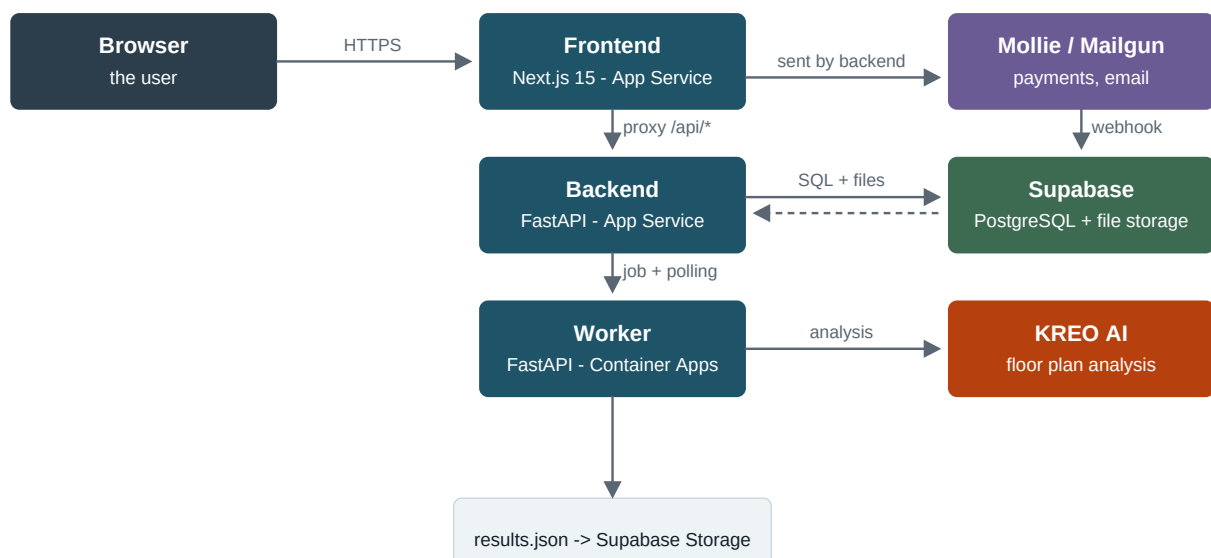
- 1 **Upload.** The user picks a PDF on /uploaden. A JPEG or PNG photo of a drawing also works: the backend straightens it and converts it to a PDF first.
- 2 **Crop and scale.** They crop to the relevant part and set the scale, meaning how many pixels equal one real-world metre. Without this the AI has shapes but no sizes.
- 3 **Analysis starts automatically.** Upload and analysis are one combined action. The original documentation described them as two steps; that is one of the corrections.
- 4 **The job goes to the worker.** The backend does not analyse anything itself. It posts the file to a separate service and receives a **job token**, effectively a receipt number.
- 5 **The worker calls the AI.** It prepares the image, sends it to the vendor, and waits. Up to ten minutes.
- 6 **The backend polls.** Every few seconds it asks the worker whether the job is finished. It gives up after fifteen minutes and marks the job failed.
- 7 **Results are stored.** The worker writes a `results.json` file listing every room, wall, door and window as coordinates plus an area.
- 8 **The viewer draws them.** The browser renders the PDF and overlays coloured outlines with the square metres labelled. The user pans, zooms, and clicks rooms to include or exclude.
- 9 **A quote is generated.** The firm's price list turns areas into money, and the result becomes a PDF.
- 10 **The quote is sent.** Downloaded, or emailed to the client with the PDF attached.

The one non-obvious part. Nothing notifies the backend when analysis finishes. It repeatedly asks. That polling loop is why a worker restart loses jobs, and it is the root of several rough edges in question 8.

QUESTION 2

The tech stack and architecture

Three programs running on three separate machines, talking over HTTP. The split exists so that a ten-minute analysis cannot block everyone else's clicks.



Solid arrows are requests; the dashed arrow is the backend reading results back. The browser only ever talks to the frontend's address.

The three services

Service	Built with	Responsible for	Runs on
Frontend	Next.js 15, React 19, TypeScript	Everything the user sees, including the interactive viewer	App Service, metenai-frontend
Backend	Python 3.11, FastAPI	All business logic: login, database, billing, quote PDFs. 17 route modules	App Service, metenai-api
Worker	Python 3.11, FastAPI, Docker	Only the AI analysis pipeline. One file, about 700 lines	Container Apps, metenai-worker

Why split at all. A web server serves many users by answering each request quickly. An analysis takes up to ten minutes; running that inside the backend would tie up a worker thread and everyone else would wait. Moving it to its own service puts the slow work where it blocks nothing. The second benefit is operational: each service can be restarted or updated on its own.

How they talk. The browser only knows the frontend's address. Next.js quietly forwards anything beginning `/api/` to the backend, so the backend's address never appears in the browser. The backend reaches the worker over an internal URL.

Libraries worth recognising

Library	Where	What it does
FastAPI, Uvicorn	Backend, worker	The web framework, and the server that runs it
SQLAlchemy	Backend	Lets you use database rows as Python objects instead of writing SQL
Alembic	Backend	Versioned scripts that change the database structure. There are 40
PyMuPDF (fitz)	Backend, worker	Reads PDFs: extracts pages, rotates, converts to images
Playwright	Backend	Runs an invisible Chromium to turn quote HTML into a PDF
bcrypt, python-jose, pyotp	Backend	Password hashing, login tokens, two-factor codes
httpx / requests	Both	Making HTTP calls out to Supabase and the AI vendor
PDF.js	Frontend	Renders the PDF inside the browser viewer
Tailwind CSS, SWR	Frontend	Styling, and keeping fetched data fresh

QUESTION 3

How the codebase is organised

One repository holds all three services - a monorepo. One clone, one branch, and a change touching backend and frontend is a single commit.

Folder	Contents
backend/	The FastAPI API. See the breakdown below

Folder	Contents
frontend/	The Next.js app: <code>src/app/</code> for pages, <code>src/components/</code> for 19 shared components, <code>src/lib/</code> for the API helpers and translations
worker/	One file, <code>app/main.py</code> , plus a Dockerfile
infra/	Docker Compose for local development, and a Terraform skeleton that is not used
docs/	Documentation, including the two files listed below
.github/workflows/	Seven CI/CD pipelines
CLAUDE.md	The project's coding rules: Dutch for user-facing text, the vendor-name rule

Inside backend/app/

Folder	What is in it
api/v1/	17 route modules, one per domain: auth, upload, floor_plans, quotes, subscription, credits, team, admin, referrals, tutorial
models/	20 SQLAlchemy models - one Python class per database table
services/	10 modules of shared logic: email, credits, the analysis poller, quote numbering, payment reconciliation
core/	5 utilities: config, authentication, security middleware, Supabase and Mollie clients
alembic/versions/	40 migration scripts, 0001 to 0040

Read these two first. `docs/FEATURE_STATUS.md` tracks all 64 features across four phases and is the definitive answer to what is actually built. `docs/CHANGES.md` is a session-by-session log of why things are the way they are. There is also an undocumented Sphinx documentation tree in `docs/source/` that the technical document never mentions.

QUESTION 4

The Azure environment

Everything lives in one resource group, `rg-metenai-prod`. Two App Services run the backend and frontend, a Container App runs the worker, a registry stores its images, and a Key Vault holds every secret. Staging duplicates all three services in the same resource group.

Resource	Name	What it is
App Service	<code>metenai-api</code>	The backend, on a Python 3.11 runtime
App Service	<code>metenai-frontend</code>	The frontend, on Node.js 22
App Service Plan	<code>asp-metenai-prod</code>	The machine both App Services share. Basic B1 - a small tier, which is why cold starts matter
Container App	<code>metenai-worker</code>	The worker. Scales to zero when idle, so it costs nothing between analyses
Container Apps Env.	<code>cae-metenai-prod</code>	The environment the Container App runs in

Resource	Name	What it is
Container Registry	metenai-registry	Private store for the worker's Docker images
Key Vault	kv-metenai-prod	Every production secret: database URL, API keys, JWT secret
Staging	...-staging	Parallel copies of all three services, in the same resource group

Two mechanisms worth understanding

Key Vault and managed identity. Secrets are never typed into Azure's configuration screens. The configuration holds a reference such as `@Microsoft.KeyVault(VaultName=kv-metenai-prod;SecretName=mollie-key)`, which Azure resolves at startup and injects as an ordinary environment variable. The app proves it is entitled to ask through a **managed identity** - an identity Azure gives the running service itself, so there is no password anywhere to steal.

Scale to zero, and the cost of it. The worker shuts down completely when no analysis is running. The App Services do not, but the Basic tier unloads an idle app after about twenty minutes, making the next visitor wait through a slow start. A scheduled job called `keep-warm.yml` pings production every fifteen minutes purely to prevent this. It is a workaround for the hosting tier, not a feature.

QUESTION 5

Access you will need

Seven systems, listed below. The documentation names every resource precisely but never says who grants access to them - so treat this as the list to take to whoever owns the accounts, not as a set of instructions.

System	What you need	What it unlocks
GitHub	Repository access, and the ability to push to development and staging	The code, and the deployment pipelines. Note that pushing to development deploys to production
Azure portal	Reader at minimum on <code>rg-metenai-prod</code> ; Contributor to deploy	Logs, configuration, restarts, and running the manual worker deployment
Azure Key Vault	Get permission on <code>kv-metenai-prod</code>	Reading production secrets. Often deliberately withheld from new joiners
Supabase	Project access	The database and the uploads bucket
KREO	An API key	Running an analysis at all. Without it, analysis is the one feature you cannot test
Mollie	A test API key	Payment flows. Test keys are safe; live keys are not
Mailgun	An API key and the sending domain	Any email: verification, invitations, quote delivery

You can start without most of them. The application runs locally with no Mollie, Mailgun or KREO credentials at all - those features simply fail when invoked. Set `DATABASE_URL` to a SQLite file and you need no database server either. That is enough to read the code, run the interface, and follow a request through the system.

QUESTION 6

Where and how customer data is stored

Two places, both at Supabase: a PostgreSQL 17 database in Ireland for structured records, and an uploads bucket for the PDFs, results and logos. Customers are kept apart by a `tenant_id` column, enforced in application code.

The database

Twenty tables, defined as twenty Python classes. The backend connects over an encrypted connection and keeps a **connection pool**: five connections held open, up to ten more under load, because opening one is slow. Grouped by purpose:

Area	Tables	Holds
Identity	users, tenants	Accounts, password hashes, two-factor secrets; the customer organisations
Analysis	floor_plans, analysis_jobs, analysis_feedback	Uploaded plans with scale and crop, job status and tokens, quality ratings
Billing	subscriptions, credit_ledger, payment_events	Plans and renewal dates, credits used and bought, every Mollie transaction
Quotes	quote_settings and its inclusions, exclusions and discounts; quotes, quote_rooms	The firm's price template, and each issued quote with its priced lines
Organisation	folders, referral_invitations	Grouping of plans; referrals
Admin	audit_log, activity_log, account_deletion_reasons	Who did what, the per-customer event stream, why customers left

The files

Path	Contents
user_{user_id}/{uuid}.pdf	Uploaded floor plans
analysis/{job_token}/results.json	The AI results for one job
logos/{user_id}.{ext}	Customer company logos

Files are never made public. Downloads use **signed URLs** - temporary links the backend generates on demand and which expire. Uploads use a **service role key**, a master credential that bypasses every access rule, which is why it must never reach the browser.

Read this part twice. The intended design had two independent layers keeping customers apart: application code filtering every query by `tenant_id`, and PostgreSQL's Row-Level Security refusing to return other tenants' rows even if the code asked wrongly. In production the second layer is inactive: the backend connects through a database role carrying `BYPASSRLS`, so those policies do not apply to its queries. Isolation therefore rests entirely on every route handler remembering to filter. One forgotten filter is a cross-customer data leak with nothing behind it to catch the mistake. The documentation states this openly and records the fix as tracked but incomplete.

QUESTION 7

Testing and release process

No automated testing is described anywhere in the documentation. No test framework, no test folder, no test step in any of the seven pipelines. The word "test" appears only in reference to Mollie's test API keys. Releasing is a push to a branch.

This is worth raising early rather than assuming it is an omission in the write-up. It also makes the staging environment more important than it would otherwise be: it is the only safety net between a change and a paying customer.

The three branches

Branch	Deploys to	Note
development	Production	Despite the name. A push here reaches real customers within minutes
staging	The staging copies	Where changes should be validated first
main	Nothing	A stable release branch, updated by manual merges at milestones

Nothing is promoted automatically. A change reaches staging or production only because someone pushed to that specific branch.

The seven pipelines

Workflow	Fires on	Does
azure-backend-deploy.yml	development	Installs dependencies and Chromium, deploys the backend
azure-backend-staging-deploy.yml	staging	The same, plus runs database migrations against staging first
azure-static-web-apps.yml	development (any file)	Builds and deploys the frontend
azure-frontend-staging-deploy.yml	staging	The same, pointed at the staging backend
azure-worker-deploy.yml	development	Builds the Docker image and pushes it. Does not deploy it
azure-worker-staging-deploy.yml	staging	Builds, pushes and deploys automatically
keep-warm.yml	Every 15 minutes	Pings production to prevent cold starts

Two traps in that table. The production worker pipeline builds the image and stops - somebody must run `az containerapp update` by hand, so a green tick does not mean production changed. And only staging runs migrations automatically, so a production deploy that expects a new database column fails unless someone applied it first.

QUESTION 8

The components that are harder to understand

In rough order of difficulty: the worker's threading and coordinate maths, the viewer component, tenant isolation, and the split between what deploys automatically and what does not.

1. The worker's threading model and coordinate maths

The rest of the codebase is asynchronous; the worker uses old-fashioned threads, because the calls to the AI vendor use a library that blocks. You cannot await a blocking call, so each search type gets a real thread: one per search, one watching for the project to become ready, and one that waits for the others, merges the results and uploads them.

On top of that sits a coordinate problem. The user crops in browser pixels; PyMuPDF measures from a different origin and ignores rotation; the AI returns coordinates relative to the cropped image. The worker converts between all three, and the inverse transform at the end is what makes the outlines line up over the original page. Getting any of it wrong produces shapes in visibly the wrong place.

- **Job state lives in a Python dictionary in memory.** A worker restart loses every in-flight analysis. There is no queue and no database record to recover from.
- **A timed-out search fails silently.** If one search type exhausts its budget, the thread simply ends. The job is still marked done and the missing piece shows up only in an errors dictionary. A partial result can look complete.
- **Two vendor workarounds are baked in:** photo-derived PDFs are converted to PNG because they hang at the vendor otherwise, and submission is retried for ten minutes because the vendor's "ready" signal is unreliable.

2. The viewer component

`FloorPlanViewer.tsx` is the most complex file in the codebase. It renders the PDF with PDF.js and draws mathematical shapes over it, keeping them aligned through pan and zoom, labelling areas at the centre of each room, handling click-to-select, exporting to Bluebeam Revu at 300 DPI, and saving its own state back to the backend. Several hard problems in one file.

3. Tenant isolation

Not conceptually difficult, but easy to get wrong in practice and unforgiving when you do. See question 6. The thing to internalise: there is no second layer catching a missing filter.

4. What deploys itself and what does not

The asymmetry between staging and production - staging deploys the worker and runs migrations, production does neither - is the kind of detail that costs an afternoon the first time you meet it.

Smaller things that will confuse you

Oddity	Detail
Four settings are documented but hardcoded	Session lifetime, remember-me duration, lockout threshold and lockout minutes appear as environment variables marked "currently hardcoded". Setting them changes nothing
One route is singular	POST /api/floor-plan/upload - every other floor plan route is plural
Profile updates live in the wrong module	They are in <code>plans.py</code> , not <code>auth.py</code> , which owns everything else about identity
Quotes are built in the browser	The frontend produces finished HTML and the backend only renders it to PDF - the reverse of how the rest of the system works
The architecture diagram is missing	Section 3.1 of the source has a heading and no content, in both the Word and PDF versions
Roles cannot be changed after invitation	No endpoint exists. You remove the person and invite them again

Oddity	Detail
The frontend Dockerfile is unused	Referenced only in a commented-out block
Terraform is a skeleton	Infrastructure was provisioned by hand, so the files do not describe what is running

Glossary

The terms that most often block a first read of the source document.

Term	In plain terms
API / REST / endpoint	Addresses another program can call. A URL names a thing and the verb says what to do: GET read, POST create, PUT update, DELETE remove. One such address is an endpoint
Middleware	Code that runs on every request before it reaches its destination. Used here to check you are logged in, so each endpoint does not have to
ORM	Object-Relational Mapper. Lets you treat database rows as Python objects instead of writing SQL. A <code>User</code> class is the user's table
Migration	A versioned script that changes the database's structure. Run in order, they take an empty database to the current shape
Asynchronous	Work handed off to finish later, without the caller waiting. The analysis is asynchronous: you get a token now and the result minutes later
Polling	Repeatedly asking whether something is done, because nothing will tell you. A loop with a sleep
Webhook	The reverse: an outside service calls you when something happens. Mollie calls the backend when a payment clears
JWT	A signed, tamper-proof token proving who you are. The server verifies the signature rather than storing sessions
HttpOnly cookie	A cookie JavaScript cannot read, so a malicious script on the page cannot steal the login token
Container / Docker	A program packaged with its dependencies so it runs identically anywhere. Like a virtualenv that also includes the operating system libraries
Rasterize / DPI	Turning a vector drawing into pixels. At 300 dots per inch an A4 page becomes roughly 2480 by 3508 pixels
Multi-tenant / tenant	One running copy of the software serving many customer organisations, kept apart in code. Each organisation is a tenant
RLS	Row-Level Security. A PostgreSQL feature where the database itself refuses rows you should not see. Currently bypassed here
CI/CD	Automation that builds and ships code when you push to a branch

Source: Meten.AI Technical Documentation, Inge Lodder and Raul Quintana Gonzalez, 26 June 2026, read with tracked changes resolved. Where a correction contradicts the original text, the correction is followed here.